WHAT IS COMPUTER PROGRAMMING?

In general, computer programming can be seen as a process of

- ***Designing***
- ***Writing***
- ***Testing***
- ***Troubleshooting*** & ***Debugging***
- ***Documenting***
- ***Maintaining***

computer programs[1].

In general, the programming process can be seen as a ***problem solving process*** where the programmer should do the following:

1	***Understand the problem***	Focus on the problem. Analyze the problem. Refine the problem by decomposing it into sub-problems.
2	Devise a plan	**Design the program**
3	Carry out the plan	**Write the program code in a programming language**
4	Evaluate the results	**Test the program** for errors and quality/usability etc.

Three views on programming vs learning programming (Booth 2001)

1. Programming as ***coding***

   The focus is upon the ***computer and the mechanics of producing program code***.

   ☞ Learning to *use a programming language and programming tools*[2].

2. Programming as ***problem-solving***

   The focus is upon ***the problem to be solved***, not the computer and mechanics.

   ☞ Learning to *solve problems by programming*.

3. Programming as ***social activity***

   The focus is upon the computer program as a ***product, to be used by other people***.

   - Make the program so that its code can be *easily read*, *understood*, and *modified* by other programmers
   - Make the program *easy and efficient for end-users to use*.

   ☞ Learning the *good programming practice* and to write *high quality* programs.
   - Here the scope also involves *usability* and *user experience*.

[1] The process often involves the ***reuse*** of already existing software components, programming knowledge (*generic algorithms etc.*), and software knowledge (*design patterns etc.*).

[2] Typically, the programmer uses an *integrated development environment* (**IDE**) that provides comprehensive facilities to computer programmers for software development. Modern IDEs consists of a ***source code editor, intelligent code completion***, ***dynamic help features***, ***build automation tools***, and a ***debugger***.

Program A computer program is a sequence of ***instructions*** written in a ***programming language*** to perform a specified task for a computer.

Source code Source code is a collection of ***human-readable*** instructions written in a programming language.

A source code written in a *scripting language* (e.g. JavaScript, PHP, Python etc.) can be run directly from the source code using a script execution engine. In the case of ***JavaScript*** the script execution engine (JavaScript engine) is included in the web browser. The JavaScript engine ***interprets*** the source code written in JavaScript. That is, you can run the program without any further steps after you have created or modified the source code.

A source code written in a *compiled language* (e.g. **Java**, C#, C++, C etc.) must be first ***compiled*** before the program can be executed. The compiler translates the human-readable source code into *machine-readable executable instructions*. The compiler saves the result of the compilation as *non-human readable* binary code ('ones and zeros').

Programming language

Programming languages are artificial languages used to write instructions that can be either directly interpreted and executed by the computer or compiled into machine-readable instructions and then executed by a computer.

- Set of formal specifications concerning ***vocabulary***, ***syntax***, and ***semantics*** (meaning).
- Used to describe the ***process*** and ***data structures***.

For ***describing the process***, the very basic instructions provided by a programming language usually cover the below.

Input		Get data from the user (or, from a database, file or device)
Output		Display data (or, write data to a database, file or device)
Simple action		Arithmetic operation Assignment operation (*save a value to a variable*) String handling operation (*'string' means a string of characters*)
Program flow control	**Conditional execution**	Check for a certain condition to decide if a certain set of actions should be executed or not.
	Repetition	Perform a certain set of actions repeatedly as long as a certain condition is met.
	Making a call to a subroutine	Perform certain set of actions by executing a certain subroutine that contains the actions.
	Exception handling	Change the program flow by jumping to an exception handler when an error occurs.

Algorithm In computer science, an algorithm is a ***set of exactly specified ordered steps suitable for implementation as a computer program*** for solving a problem.

Variables

IMPORTANT! The concept of variable in ***computer science*** is different from the concept of variable in ***mathematics***.

In the programming context, a ***variable is a named container where the program can store a single value***. The program can change the value held in a variable many times. When the program execution ends, all its variables are discarded.

Why variables are needed?

During the program execution, the inputted values and the computed values should be stored in the computer's main memory.

For example, a program ***inputs values from the user*** (e.g. gross salary and income tax percentage) and then ***computes a new value*** (net salary) from the inputted values. Finally, the program displays the result of the computation (net salary). Typically, we need three variables to store these three different values.

In a program, you create a new variable by *declaring* it. In JavaScript, you can declare variables by using the **var** keyword as below. Please notice the use of semicolons below.

```
var grossSalary;
var taxPercentage;
var netSalary;
```

You can think that each variable is a named box where you can store a single value.

You can set/change the value of a variable with the ***assignment*** operation as below. Please notice that in JavaScript '**=**' is the *assignment operator*. It is *not* a comparison operator.

```
grossSalary = 2000;
taxPercentage = 25;

netSalary = grossSalary - (grossSalary * taxPercentage / 100);
```

To make this simple example complete, we print the result to the browser's console window.

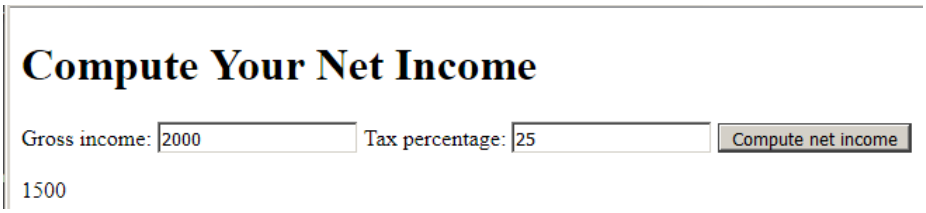
```
console.log(netSalary);
```

Program size (number of source code lines) vs the challenge

Number of code lines

10	→	trivial
100	→	basic programming skills emphasized
1 000	→	program design skills emphasized
10 000	→	requirement analysis and design specification skills emphasized
100 000	→	systems development skills and methodology emphasized
1 000 000	→	Firefox web browser
10 000 000	→	Operating systems, thousands of developers might be involved
50 000 000	→	Windows 7
100 000 000	→	Debian GNU/Linux 4.0, written in the C programming language

A COMPLETE JAVASCRIPT PROGRAM EXAMPLE



OSE_ComputeNetIncome.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8" />
    <title>Compute Net Income</title>
</head>
<body>
    <h1>Compute Your Net Income</h1>

    <label>Gross income: </label>
    <input type="text" id="txtGrossIncome" />

    <label>Tax percentage: </label>
    <input type="text" id="txtTaxPercentage" />

    <input type="button" onClick="computeNetIncome()" value="Compute net income" />

    <p id="pOutput"></p>

    <script src="OSE_ComputeNetIncome.js"></script>
</body>
</html>
```

OSE_ComputeNetIncome.js

```
function computeNetIncome() {

    var grossIncome;
    var taxPercentage;
    var netIncome;

    grossIncome = Number(document.getElementById("txtGrossIncome").value);
    taxPercentage = Number(document.getElementById("txtTaxPercentage").value);

    netIncome = grossIncome - (grossIncome * taxPercentage / 100);

    document.getElementById("pOutput").innerHTML = netIncome;
}
```

The JavaScript file is linked to the HTML file by using the <script> tag in the HTML code.

In the JavaScript code , we define a JavaScript function called *computeNetIncome* that should be executed when the user clicks the HTML button.

Inside the JavaScript function, we

1. Declare three variables. This time, without any initial values.
2. Retrieve values from the HTML text boxes (whose ids are *txtGrossIncome* and *txtTaxPercentage*) and convert the values to numbers before assigning them to the variables *grossIncome* and *taxPercentage*.
3. Calculate net income and assign the result to the variable *netIncome*.
4. Copy the value of the variable *netIncome* to be the content of the HTML element whose id is *pOutput*.